Deep Learning

Computer Vision

photo: fisher-price.com

Xuanji Yu

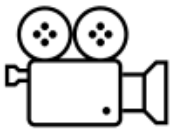
Courtesy of Professor Roch, Chapter 25, Russell & Norvig (we only cover highlights)

MIT How to AI (Almost) Anything, Paul Liang, MIT

Data: High Complexity and Different Modalities

Sensory modality

Modality refers to the way in which something expressed or perceived.



from a sensor

Raw Modalities
(closest from sensor)

Abstract Modalities
(farthest from sensor)

Examples:

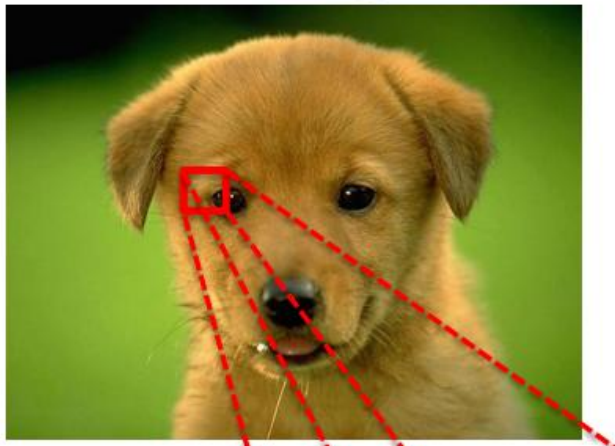
Raw Modalities		Abstract Modalities
Speech signal	Language	Sentiment intensity
Image	Detected objects	Object categories

Most of AI is about learning abstractions, or representations, from data.

Data: High Complexity and Different Modalities

Visual Modality

Color image



Each pixel is represented in $\mathcal{R}^d$, d is the number of colors (d=3 for RGB)

88	82	84	88	85	83	80	93	102
88	80	78	80	80	78	73	94	100
85	79	80	78	77	74	65	91	99
38	35	40	35	39	74	77	70	65
20	25	23	28	37	69	64	60	57
22	26	22	28	40	65	64	59	34
24	28	24	30	37	60	58	56	66
21	22	23	27	38	60	67	65	67
23	22	22	25	38	59	64	67	66

Input observation x_i

88
88
85
38
20
22
24
21
23
82
80
79
35
25
26
28
22
22
84
78
80
⋮

Binary classification problem

Dog ?

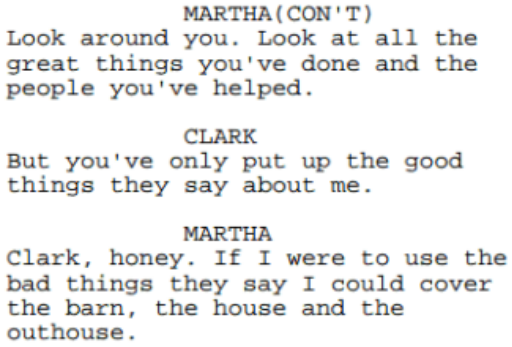
label $y_i \in \mathcal{Y} = \{0,1\}$

Data: High Complexity and Different Modalities

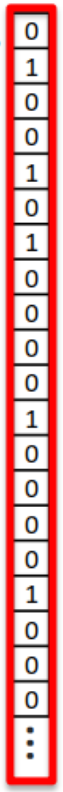
Language Modality

Written language

Spoken language



Input observation x_i



"bag-of-word" vector

$|x_i|$ = number of words in dictionary

Document-level classification

Sentiment ?
(positive or negative)

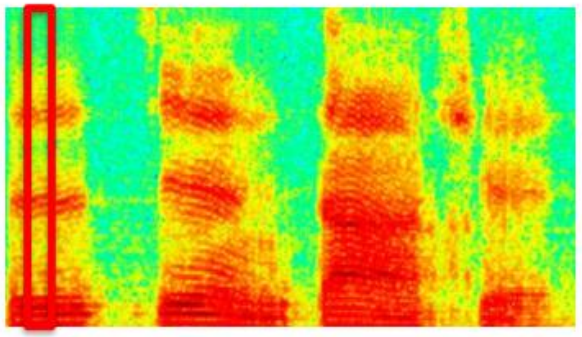
Response?

Data: High Complexity and Different Modalities

Acoustic Modality

Digitalized acoustic signal

- Sampling rates: 8~96kHz
- Bit depth: 8, 16 or 24 bits
- Time window size: 20ms
  - Offset: 10ms



Spectrogram

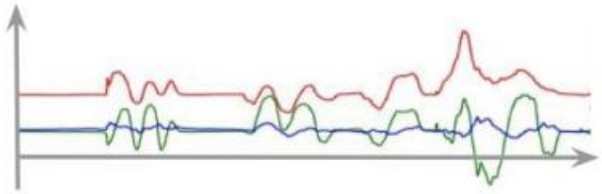
Input observation x_i

0.21
0.14
0.56
0.45
0.9
0.98
0.75
0.34
0.24
0.11
0.02

Spoken word ?

Data: High Complexity and Different Modalities

Sensor Modality



Time series data across six-axis Force-Torque sensor:
T × 6 signal.

Force-Torque Sensor

Time series data across current position and velocity of the end-effector:
T × 2 signal.

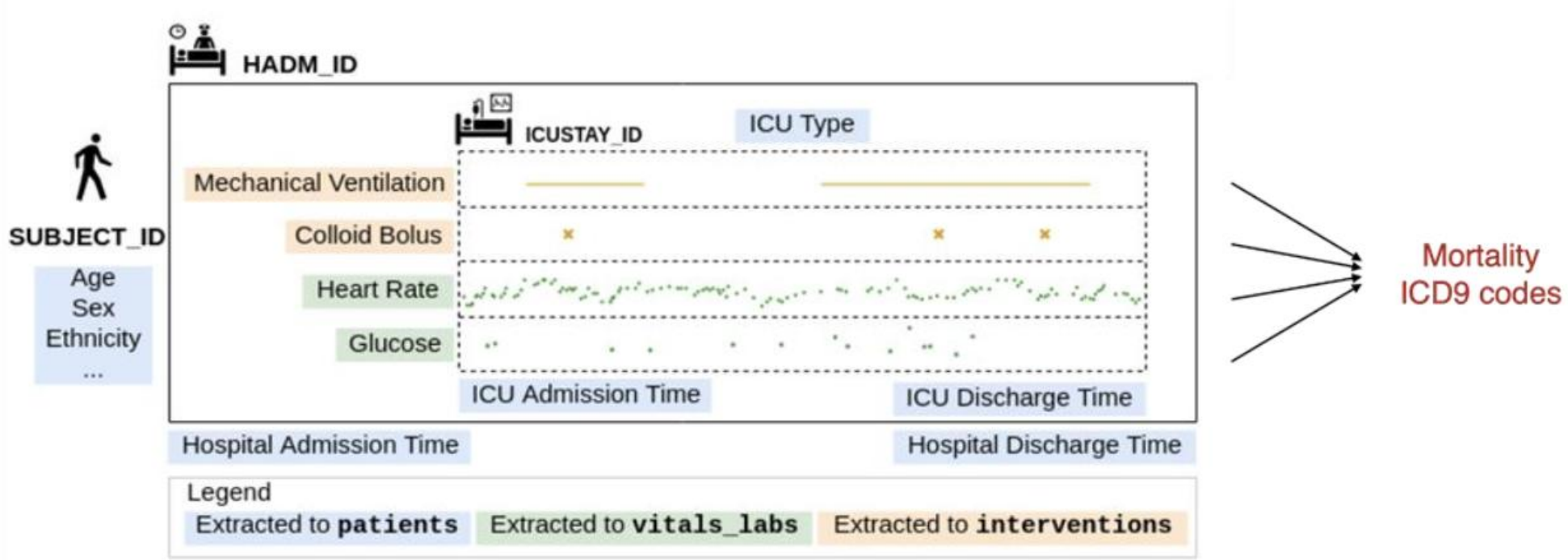
Proprioception

Measure values internal to the system (robot); e.g. motor speed, wheel load, **robot arm joint angles**, battery voltage.

Object property
Next action

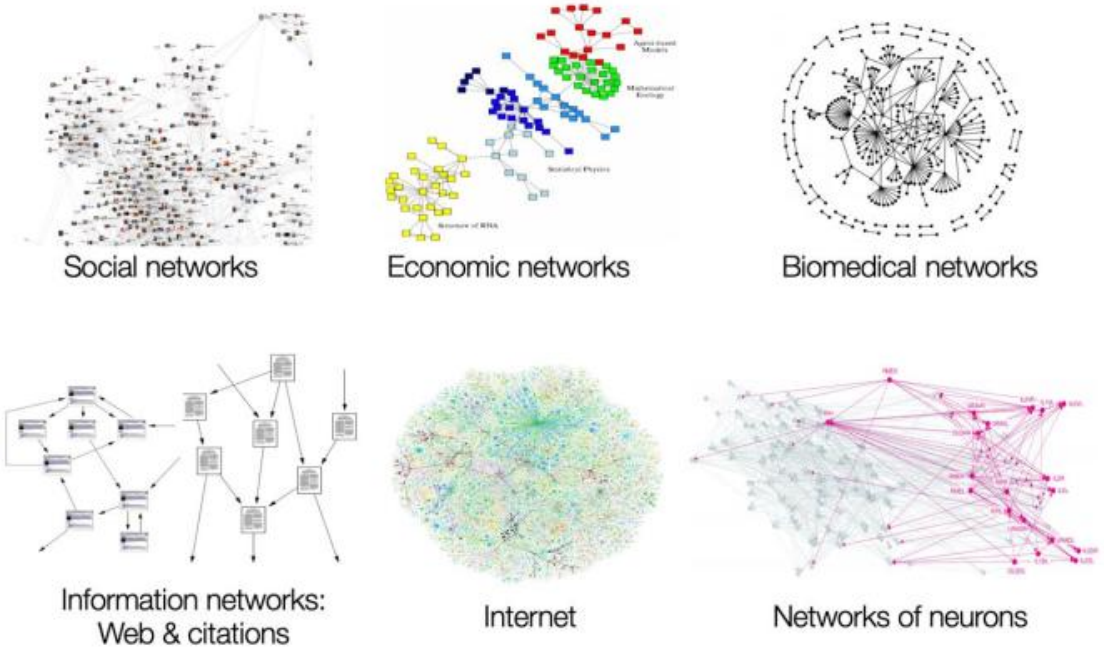
Data: High Complexity and Different Modalities

Tabular Modality



Data: High Complexity and Different Modalities

Graph Modality



Tasks on graphs:
Node classification
Link prediction
...

Using graphs:
Knowledge graphs for QA
Social network for sentiment analysis
...

Deep Learning - Deep Neural Networks

- Depth: Deeper layers typically learn more complex information
- Flexibility: model architectures can be easily adapted to different modalities

e.g., Computer Vision

Output from: You only look once (YOLO;; Redmon et al. 2016)

Our focus: Single label image classification

Spanish shawl nudibranch
(*Flabellina iodinea*)

Computer Vision Tasks

- Detection
- Labeling
- Relationships between objects

What makes this hard?

- Perspective and view point changes

mixbook.com

The same tree

Pinhole cameras & perspective

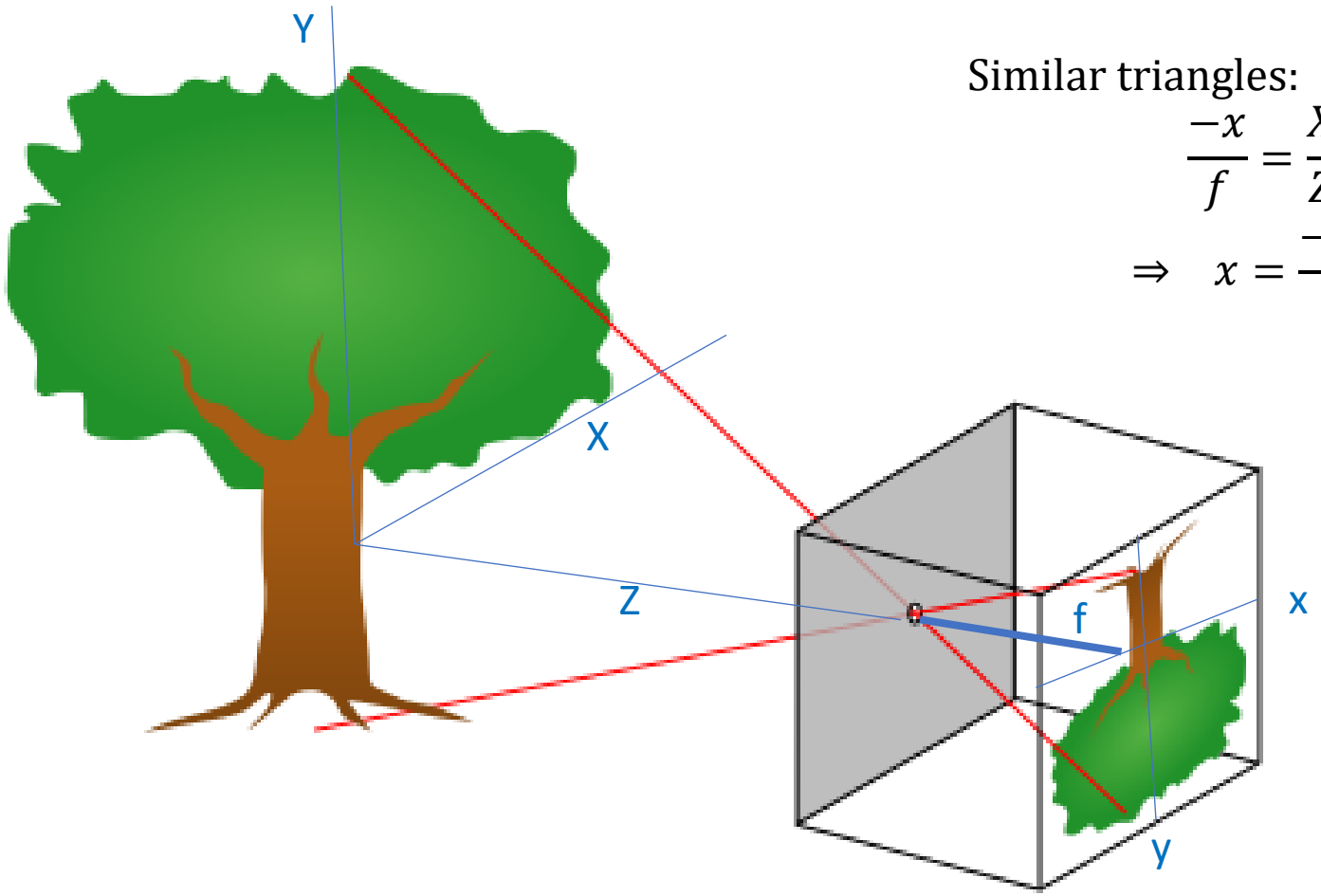


image: Wikimedia commons

Similar triangles:

$$\frac{-x}{f} = \frac{X}{Z}, \frac{-y}{f} = \frac{Y}{Z}$$

$$\Rightarrow \quad x = \frac{-fX}{Z}, y = \frac{-fY}{Z}$$

f – focal distance between pinhole & projection plane

Illumination

Each row is the same person with different lighting conditions

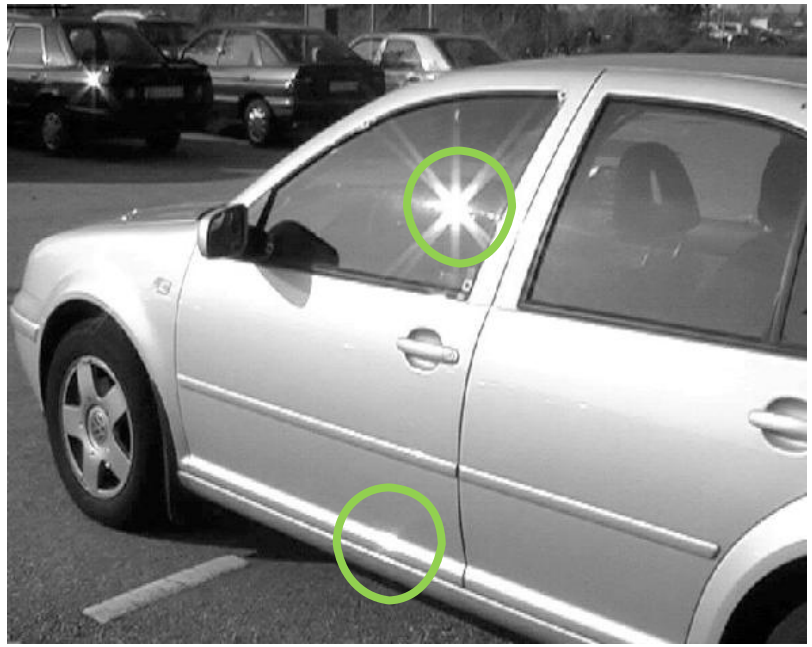


image: Aanæs 2003

Specular reflections

Deformation

Instances of the same class may be nonridgid and undergo deformation

image credit: Sarah K, Benton County 4-H, MN, https://extension.umn.edu/

Occlusion

Objects can be hidden by other objects

Clutter and noise

Background regions that are similar to the foreground

- Satantic leaf-tailed gecko
- rock wallabees

images from *Wired* and Camouflage by Steve Parker
educational fair use:

Variation

WizSmart dog pads

Images have same label (dog) although they are quite different

Summary

Computer vision must deal with a variety of complicating factors

- perspective
- illumination
- deformation
- occlusion
- clutter and noise
- variation

image credit: Nate @ http://floorsix.blogspot.com/

Like all supervised machine learning problems

- Need labeled corpus
  - Training data
  - Test data

- Simple k-nearest neighbor classifier
  - Training: Retain database of images
  - Test: Look for most similar images and use plurality class

Measuring distance between images

- Naïve method: pixel by pixel comparison (all we will be doing)

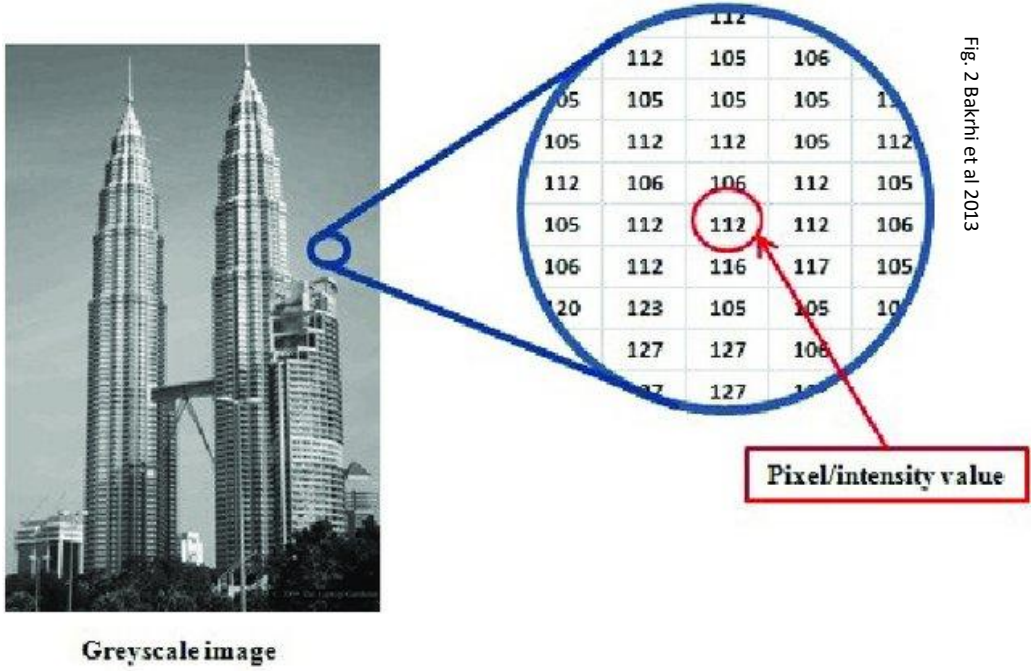
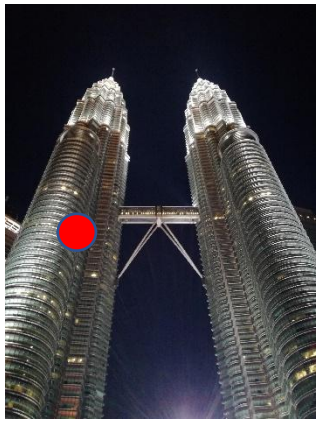


Fig. 2 Bakrhi et al 2013

- L1 or L2 distance



L_p distance: $d(I,\ J\ ) = \sqrt[p]{\sum_{x,y}|I_{x,y} - J_{x,y}|^p}$

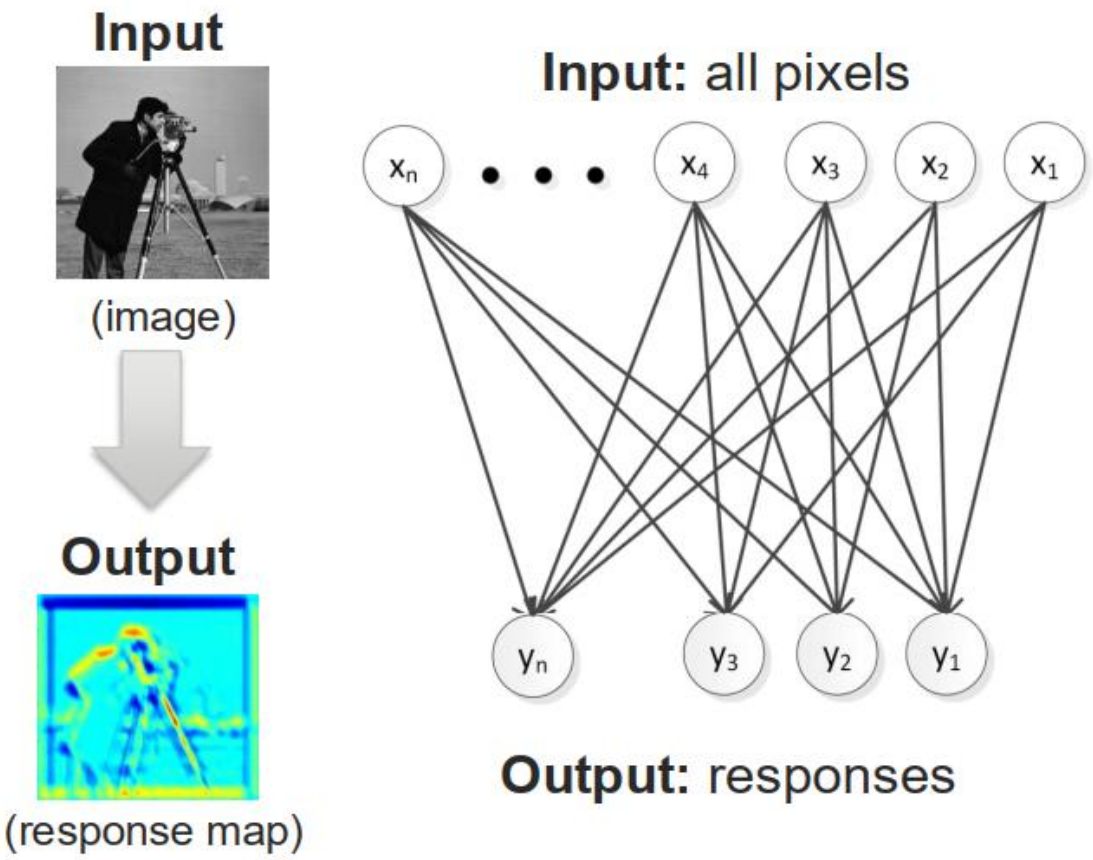
How well is our method going to do with respect to?

- perspective
- illumination
- deformation
- occlusion
- clutter and noise
- variation

images from CIFAR 100 dataset

Let’s use Neural Networks

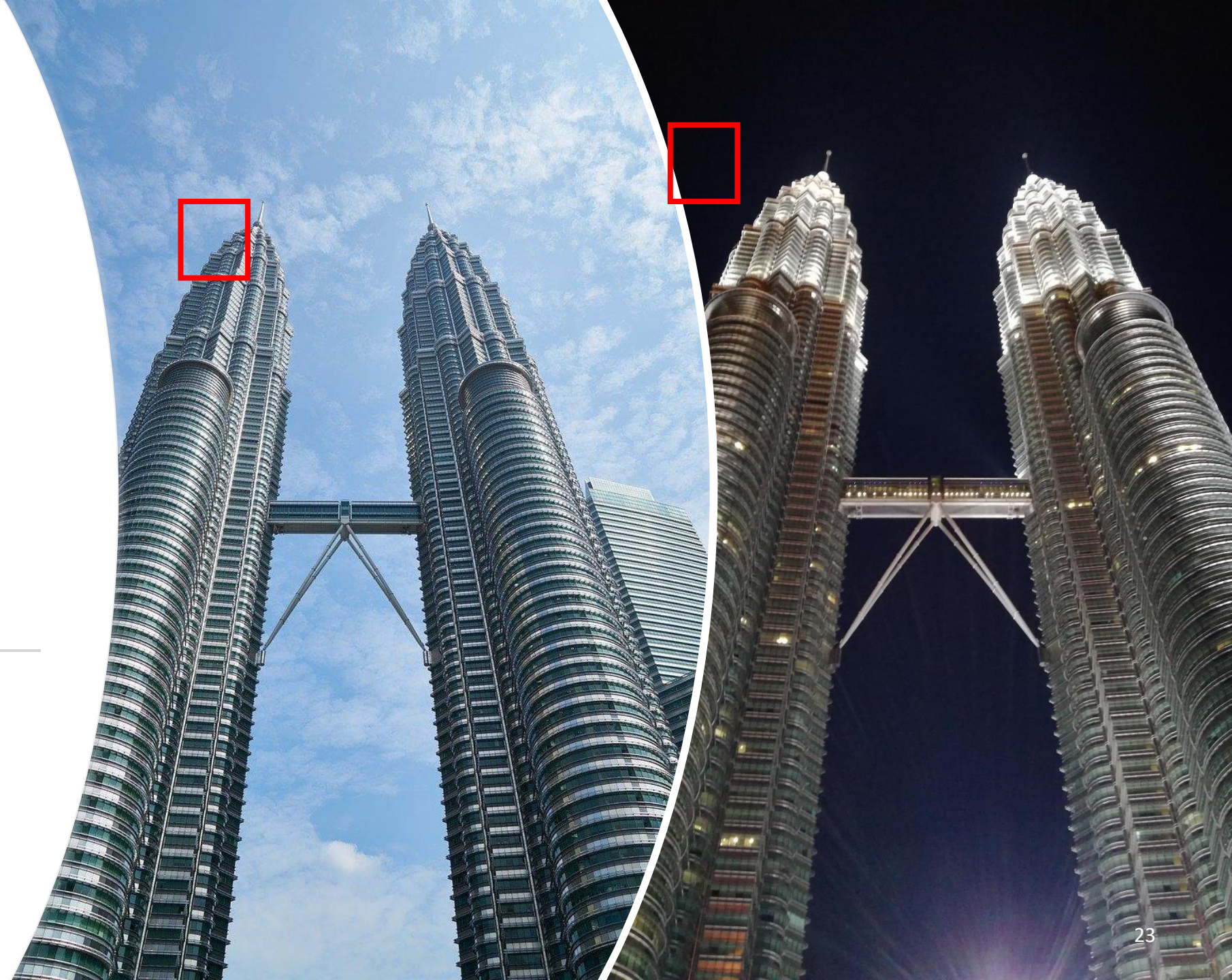
- Multi-Layer Perceptron (fully connected feedforward neural network)



- Not efficient!
  - 200 × 200 image requires 40,000 × n parameters
- Not spatial invariant!
  - may learn different outputs for different pixel positions

A better way…

What if we compared local regions?



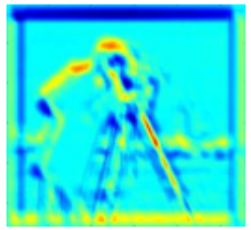
Convolutional neural networks

Modification 1: Only apply the filter to a small sliding window
-> for efficiency and locality

Input

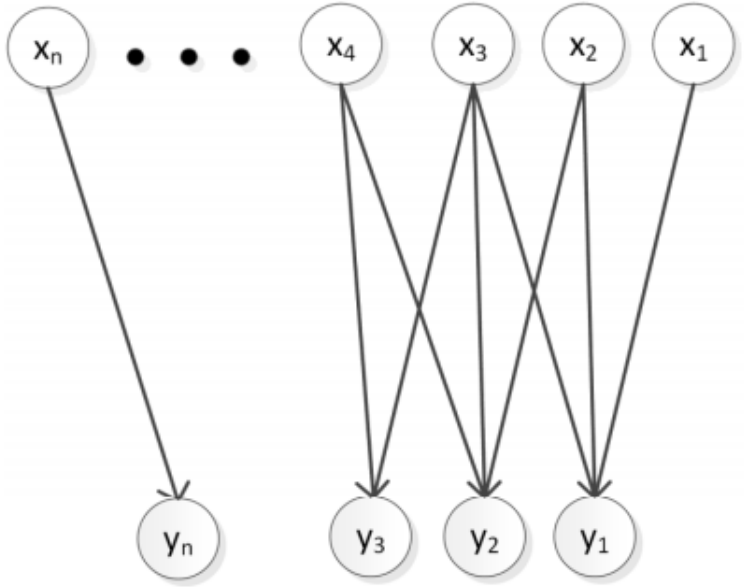
(image)

Output



(response map)

Input: all pixels



Output: responses

Example with 1D filter:

w_1	w_2	w_3

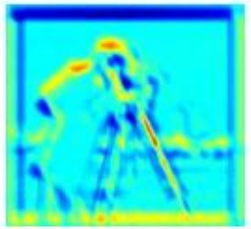
Convolutional neural networks

Modification 2: Same filter applied to all sliding windows
-> for spatial invariance

Input

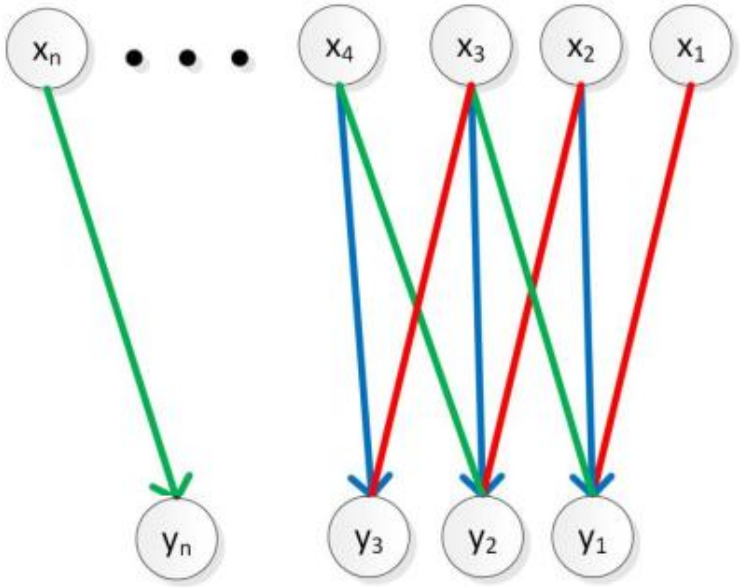
(image)

Output



(response map)

Input: all pixels

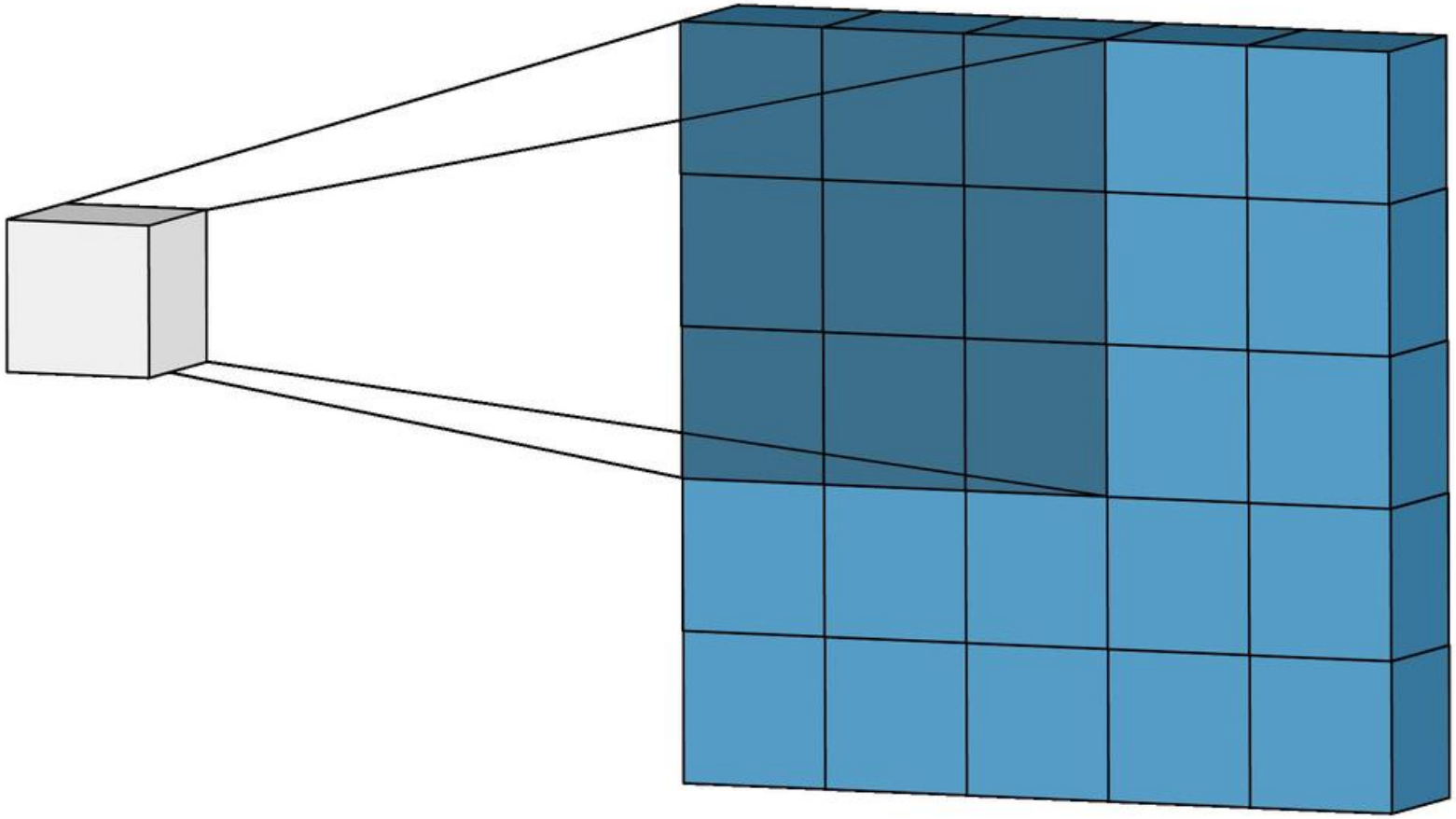


Output: responses

Example with 1D filter:

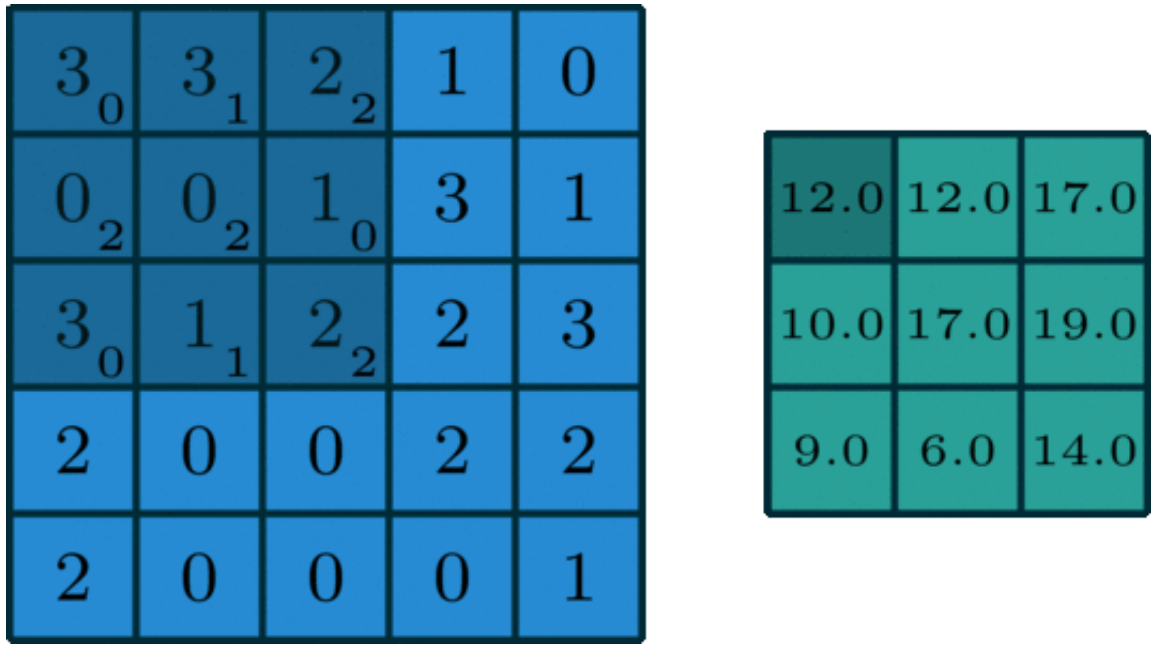
w_1	w_2	w_3

Convolution in 2D



Animation credit: Irhum Shafkat

Convolution in 2D



Convolutional filter or kernel

$$w = \begin{bmatrix} 0 & 1 & 2 \\ 2 & 2 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

Animation credit: Irhum Shafkat

Convolution in 2D

- Convolve (*) image I with kernel K:

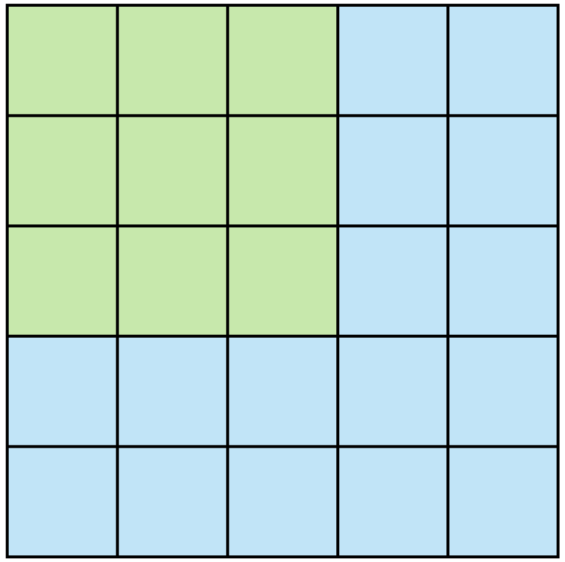
$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I[m,n]K[i-m, j-n]$$

- Many libraries implement cross-correlation instead of convolution (kernel not reversed, not important in practice):

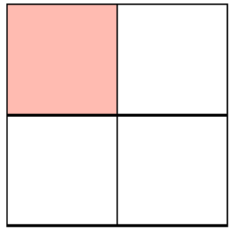
$$\sum_m \sum_n I[i+m, j+n]K[i,j]$$

Convolutional layers

- Same parameter or weights per filter (Spatial invariant)
- Fixed-size kernel, e.g., 5 X 5
- Convolutional output is passed through the activation function
- Stride parameters cause skips



Stride 2

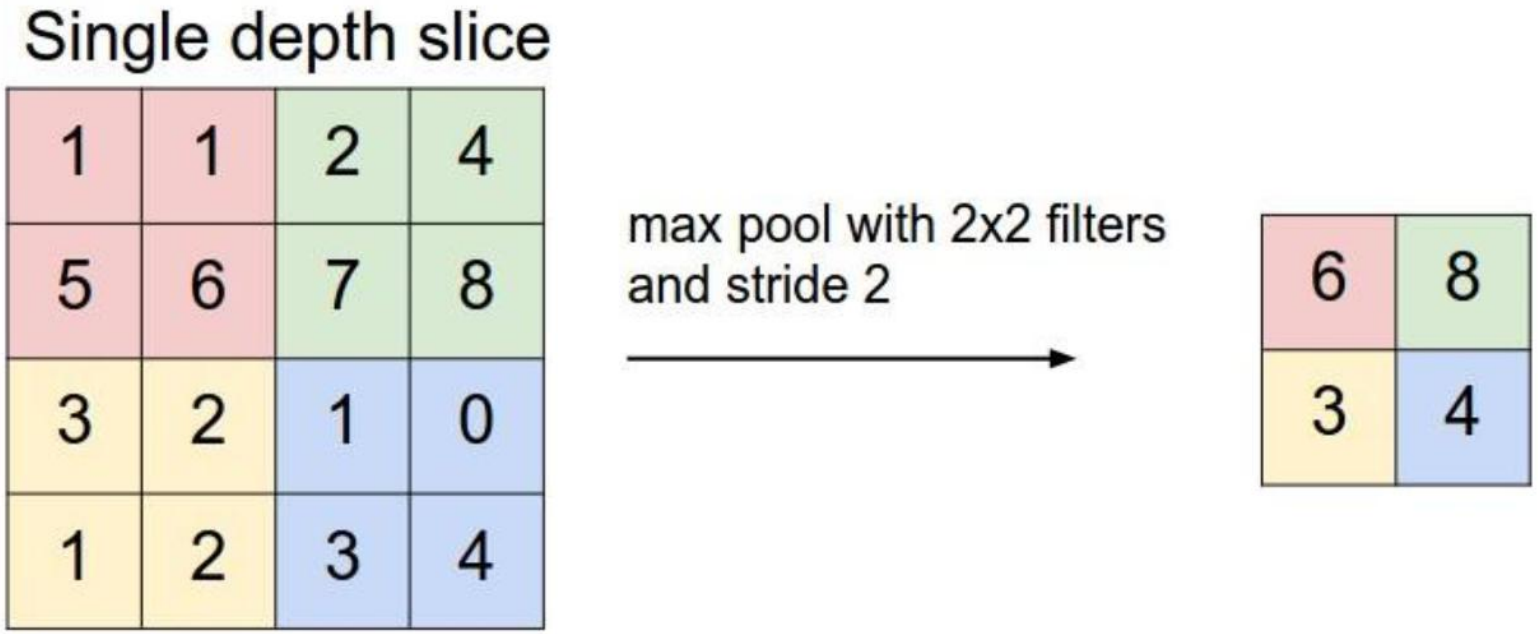


Feature Map

Pooling layers

Information aggregation:

- Common to pool regions and take some statistic, e.g., max
- Stride parameters cause skips



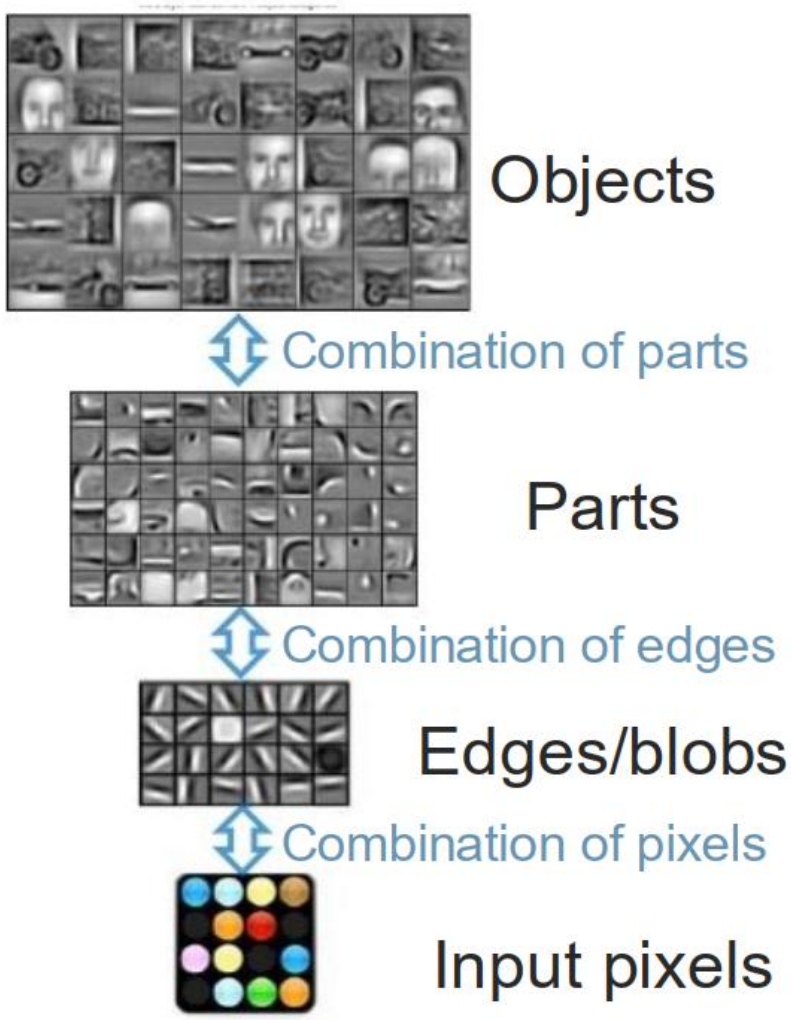
Convolutional neural networks

Multiple convolutional layers

- Allows the network to learn combinations of sub-parts, to increase complexity

Multiple pooling layers

- Allows the network to learn increasingly abstract & summarized information



Basic ideas

- Convolution is an operation like any other
- Loss can be backpropagated and kernel weights adjusted
- Loss function guides what is learned
- Downsampling operations called pooling reduce size

- Going from convolution to classification…
  - Flatten last layers
  - feed-forward network

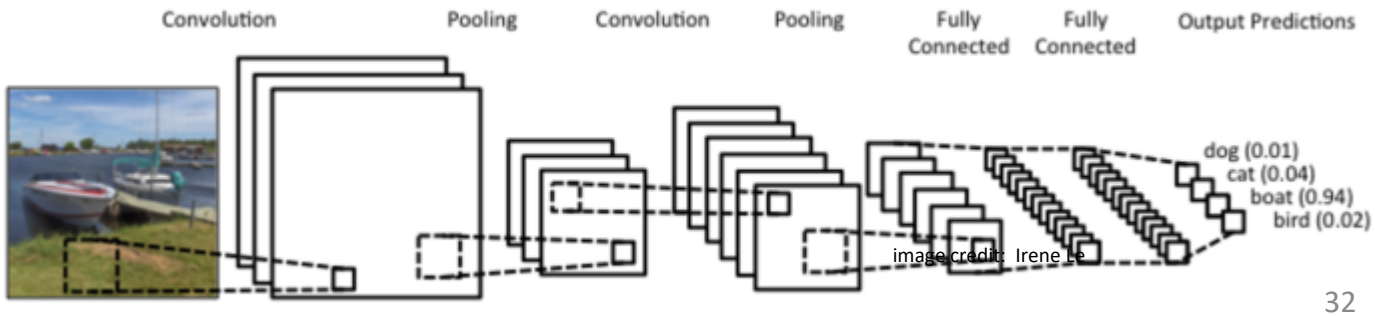


image credit: Irene Li

Keras convolutional layers

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input, Conv2D, MaxPooling2D, Flatten

model = Sequential()

# 1. Input layer for 1000x1000 grayscale images
model.add(Input(shape=(1000, 1000, 1)))  # (Height, Width, Channels)


# 2. Conv layer: 64 filters, 5x5 kernel
model.add(Conv2D(64, kernel_size=(5, 5), activation='relu'))


# 3. Max Pooling: downsample by 2x2
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
```

Keras convolutional layers

```
# 4. Second Conv layer: 32 filters, 5x5 kernel
model.add(Conv2D(32, kernel_size=(5, 5), activation='relu’))


# 5. Second Max Pooling: again downsample
model.add(MaxPooling2D(pool_size=(2, 2)))

# 6. Flatten: convert 2D feature maps to 1D vector
model.add(Flatten())

# 7. Fully connected (Dense) layer with 1000 neurons
model.add(Dense(1000, activation='relu'))

# 8. Output layer: one unit per class
model.add(Dense(NumClasses, activation='softmax'))
```

Unified View of Deep Learning Models

Composing differentiable functions and training objectives.

1. Basic representation building blocks for each element

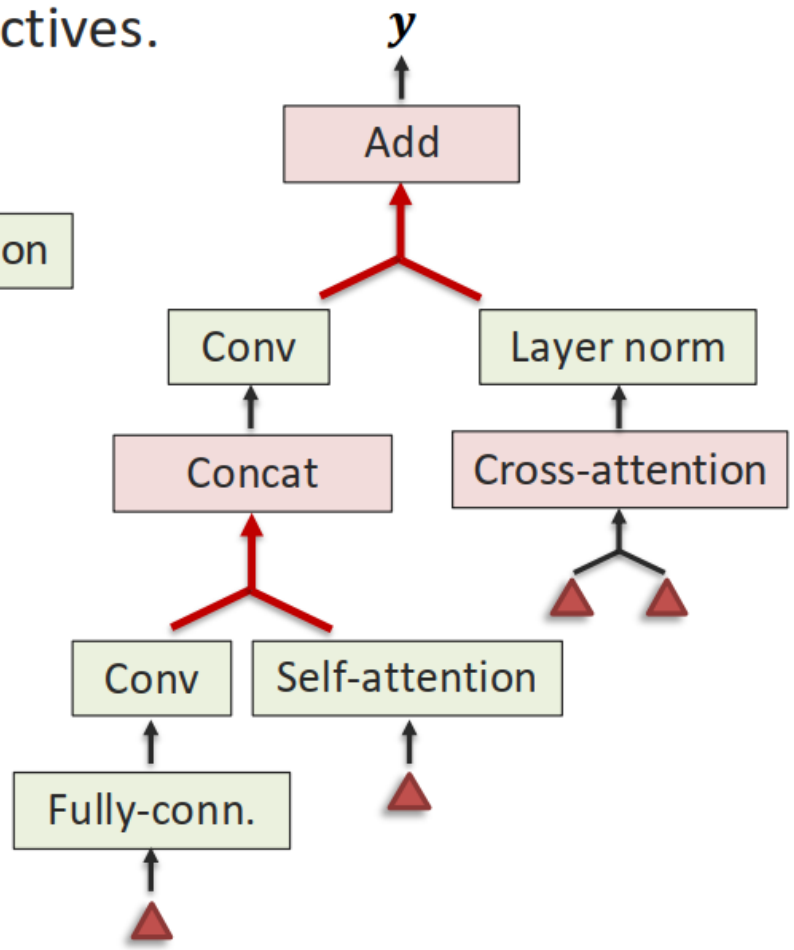
   Fully-connected | ReLU | Layer norm | Conv | Self-attention

2. Basic information aggregation blocks

   Concat | Cross-attention | Add | Max

3. Compute loss function

4. Take gradients, update with stochastic gradient descent



How to Learn: Workflow

Typical and practical workflow for building and training learning models

- Data → Baselines → Model Design → Overfit → Regularize → Tune

How to Learn: Data

Start with understanding and preparing the data.

1. Collect data: (time and cost)
   - how much data to collect
   - how much to label
2. Clean data:
   - remove duplicates, corrupt, label errors
   - normalize/standardize
   - anomaly/outlier detection
3. Identify patterns, biases, and potential preprocessing/filter steps
4. Visualize data: plot, dimensionality reduction (PCA, t-sne), cluster analysis (k-Means)
5. Split in to training, validation, test

How to Learn: Baseline

Build simple models to get a sense of how hard the task is and use as baseline.

1. Start with a simple model
   - linear classifier
   - SVM
   - Random Forest
   - Shallow MLP/CNN
   - Pretrained model (vision, language)

Tips:

- Fix random seeds for reproducibility
- Simplify by disabling augmentation and complex features
- Verify loss at initialization
- Overfit on one batch to confirm correctness

How to Learn: Model design

Choose architecture and representation based on data structure.

- Choose and design model, based on
  - Data
    - base elements
    - Data representation
    - Data structure, e.g., spatial invariance for computer vision
  - Model
    - architecture
    - capacity

Tips:

- Explore state-of-the-art architectures

How to Learn: Overfit

Train a flexible model to overfit on a very small training dataset (on purpose at first),

- showing that your model has enough capacity to learn.
- If you can’t overfit training data at all, maybe model too small or bugged.

Tips:

- Visualize predictions and loss dynamics
- Ensure gradients affect only expected inputs
- Avoid premature optimization of architecture

How to Learn: Regularize for Generalization

Now you apply regularization to control overfitting.

- Add more real data – the best way to reduce overfitting
- Use data augmentation and pretraining
- Reduce input dimensions and model size
- Techniques:
  - L1/L2 regularization (penalize large weights)
  - Dropout (for neural nets)
  - Early stopping (stop training when validation loss stops improving)

How to Learn: Tune

Fine-tune the model using hyperparameter optimization.

- Prefer random search over grid search
- Tune
  - Learning rate
  - Regularization strength
  - Number of layers/neurons
  - Batch size
  - Optimizer type (Adam, SGD, etc.)

Leave models training longer than expected

Use model ensembles